

10003 Cutting Sticks

You have to cut a wood stick into pieces. The most affordable company, The Analog Cutting Machinery, Inc. (ACM), charges money according to the length of the stick being cut. Their procedure of work requires that they only make one cut at a time.

It is easy to notice that different selections in the order of cutting can led to different prices. For example, consider a stick of length 10 meters that has to be cut at 2, 4 and 7 meters from one end. There are several choices. One can be cutting first at 2, then at 4, then at 7. This leads to a price of $10 + 8 + 6 = 24$ because the first stick was of 10 meters, the resulting of 8 and the last one of 6. Another choice could be cutting at 4, then at 2, then at 7. This would lead to a price of $10 + 4 + 6 = 20$, which is a better price.

Your boss trusts your computer abilities to find out the minimum cost for cutting a given stick.

Input

The input will consist of several input cases. The first line of each test case will contain a positive number l that represents the length of the stick to be cut. You can assume $l < 1000$. The next line will contain the number n ($n < 50$) of cuts to be made.

The next line consists of n positive numbers c_i ($0 < c_i < l$) representing the places where the cuts have to be done, given in strictly increasing order.

An input case with $l = 0$ will represent the end of the input.

Output

You have to print the cost of the optimal solution of the cutting problem, that is the minimum cost of cutting the given stick. Format the output as shown below.

Sample Input

```
100
3
25 50 75
10
4
4 5 7 8
0
```

Sample Output

```
The minimum cutting is 200.
The minimum cutting is 22.
```

Algorithm 1: Rod Cutting (recursion)

```
1: procedure SOLVE( $l, r$ )
2:   ▷ Case 1: no cuts can be made in between
3:   if  $r - l \leq 1$  then
4:     return 0
5:   end
6:
7:   ▷ State transition
8:    $\text{least} \leftarrow \infty$ 
9:    $\text{length} \leftarrow \text{cuts}[r] - \text{cuts}[l]$ 
10:
11:   for  $k \in [l + 1, r - 1]$  do
12:      $\text{cost} \leftarrow \text{length} + \text{Solve}(l, k) + \text{Solve}(k, r)$ 
13:      $\text{least} \leftarrow \min(\text{least}, \text{cost})$ 
14:   end
15:
16:   return  $\text{least}$ 
17: end
```

```
1  #include <iostream>
2  #include <climits>
3  using namespace std;
4
5  int cuts[] = {0, 4, 5, 7, 8, 10};
6
7  int Solve(int l, int r)
8  {
9      if (r - l <= 1) return 0;
10
11     int length = cuts[r] - cuts[l];
12     int least = INT_MAX;
13
14     for (int k = l + 1; k < r; ++k)
15     {
16         int cost = length + Solve(l, k) + Solve(k, r);
17         least = min(least, cost);
18     }
19
20     return least;
21 }
22
23 int main()
24 {
25     cout << Solve(0, 5);
26
27
28     return 0;
29 }
```

Algorithm 2: Rod Cutting DP (Memoization)

```
1: procedure SOLVE( $l, r$ )
2:   ▷ Base case: no cuts can be made in between
3:   if  $r - l \leq 1$  then
4:     return 0
5:   end
6:
7:   ▷ Memoization check
8:   if  $\text{memo}[l][r] \neq -1$  then
9:     return  $\text{memo}[l][r]$ 
10:  end
11:
12:  ▷ State transition
13:   $\text{least} \leftarrow \infty$ 
14:   $\text{segment\_length} \leftarrow \text{cuts}[r] - \text{cuts}[l]$ 
15:
16:  for  $k \leftarrow l + 1$  to  $r - 1$  do
17:     $\text{cost} \leftarrow \text{segment\_length} + \text{Solve}(l, k) + \text{Solve}(k, r)$ 
18:     $\text{least} \leftarrow \min(\text{least}, \text{cost})$ 
19:  end
20:
21:  ▷ Save to memo and return
22:   $\text{memo}[l][r] \leftarrow \text{least}$ 
23:  return  $\text{least}$ 
24: end
```

